

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 1 – Buổi 1

Học máy là gì?

(What is Machine Learning?)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- Giải thích sự khác nhau giữa **lập trình truyền thống** (*traditional programming*) và **học máy** (*machine learning*) bằng ví dụ đời thường.
- Phân tích một bài toán theo ba yếu tố **T – E – P** (*Task, Experience, Performance*).
- Dùng đúng các thuật ngữ: **mẫu** (*sample*), **đặc trưng** (*feature*), **nhãn** (*label*), **mô hình** (*model*), **huấn luyện** (*training*), **dự đoán** (*prediction*); đọc ký hiệu X, y, n, d.
- Phân biệt **hồi quy** (*regression*), **phân loại** (*classification*) và **phân cụm** (*clustering*).
- Mô tả bảy bước của một **dự án học máy** (*machine learning project*).

Nội dung

1. Học máy là gì?

2. Các thuật ngữ cơ bản

3. Các nhóm bài toán học máy

4. Quy trình một dự án học máy

5. Tổng kết Buổi 1.1

Phần 1

Học máy là gì?

Đặt vấn đề: viết chương trình nhận biết thư rác

Bạn được giao viết chương trình tự động nhận biết **thư rác** (*spam*) trong hộp thư điện tử.

Cách 1: tự viết quy tắc (*rule-based*)

- Tiêu đề có cụm "trúng thưởng" $\Rightarrow$ rác.
- Nội dung có hơn ba dấu "!!!" liên tiếp $\Rightarrow$ rác.
- Người gửi không có trong danh bạ **và** thư chứa đường link lạ $\Rightarrow$ rác.

Đặt vấn đề: viết chương trình nhận biết thư rác

Bạn được giao viết chương trình tự động nhận biết **thư rác** (*spam*) trong hộp thư điện tử.

Cách 1: tự viết quy tắc (*rule-based*)

- Tiêu đề có cụm "trúng thưởng" ⇒ rác.
- Nội dung có hơn ba dấu "!!!" liên tiếp ⇒ rác.
- Người gửi không có trong danh bạ **và** thư chứa đường link lạ ⇒ rác.

Chuyện gì xảy ra sau vài ngày?

- Kẻ gửi rác đổi cách viết: "tr.ú.n.g th.ư.ở.n.g", "TRUNG THUONG", "bạn đã được chọn"…
- Ta phải liên tục thêm quy tắc mới.
- Danh sách quy tắc ngày càng dài, **chồng chéo**, **khó bảo trì**; thêm quy tắc này lại làm hỏng quy tắc kia.

Viết thử bộ lọc thư rác bằng quy tắc

ch01.py — ô thu_rac_quy_tac

```python
def la_thu_rac(tieu_de):
    """Bộ lọc thư rác viết bằng QUY TẮC do con người nghĩ ra."""
    tieu_de = tieu_de.lower()                # đổi hết sang chữ thường để so sánh
    if "trúng thưởng" in tieu_de:            # quy tắc 1
        return True
    if "!!!" in tieu_de:                     # quy tắc 2: ba dấu chấm than liên tiếp
        return True
    return False                             # không khớp quy tắc nào: thư thường

cac_thu = ["Chúc mừng bạn TRÚNG THƯỞNG xe máy",
           "Lịch họp nhóm thứ Hai",
           "Khuyến mãi sốc!!! Mua ngay",
           "Bạn đã tr.ú.n.g th.ư.ở.n.g 100 triệu",   # kẻ gửi rác đổi cách viết
           "Bạn đã được chọn nhận quà miễn phí"]      # không chứa từ khóa nào
for thu in cac_thu:
    print(f"{str(la_thu_rac(thu)):5s} <- {thu}")
```

Kết quả

```
True  <- Chúc mừng bạn TRÚNG THƯỞNG xe máy
False <- Lịch họp nhóm thứ Hai
True  <- Khuyến mãi sốc!!! Mua ngay
False <- Bạn đã tr.ú.n.g th.ư.ở.n.g 100 triệu
False <- Bạn đã được chọn nhận quà miễn phí
```

- Hai quy tắc bắt đúng thư 1 và thư 3.
- Hai thư cuối rõ ràng là rác nhưng hàm trả về `False`: **không quy tắc nào khớp**.
- Mỗi chiêu mới của kẻ gửi rác ⇒ lại phải viết thêm quy tắc.

Một cách tiếp cận khác: để máy tự học

Cách 2: học từ dữ liệu (*learning from data*)

1. Thu thập **hàng nghìn thư** mà người dùng đã đánh dấu "rác" / "không rác".
2. Cho máy tính **tự tìm ra** những dấu hiệu phân biệt hai loại thư.
3. Thư mới đến: máy dùng các dấu hiệu đã học để quyết định.
4. Kẻ gửi rác đổi chiêu? Chỉ cần cho máy **học lại** với dữ liệu mới.

Điểm khác biệt then chốt:

- Con người **không viết quy tắc**, mà cung cấp **ví dụ có đáp án**.
- Quy tắc được **rút ra tự động** từ dữ liệu.
- Hệ thống **cải thiện** khi có thêm dữ liệu.

Mẹo nhỏ

Đó chính là ý tưởng cốt lõi của **học máy** (*machine learning*).

Ví dụ đời thường: dạy trẻ nhận biết con mèo

Ví dụ đời thường: Học từ ví dụ, không học từ định nghĩa

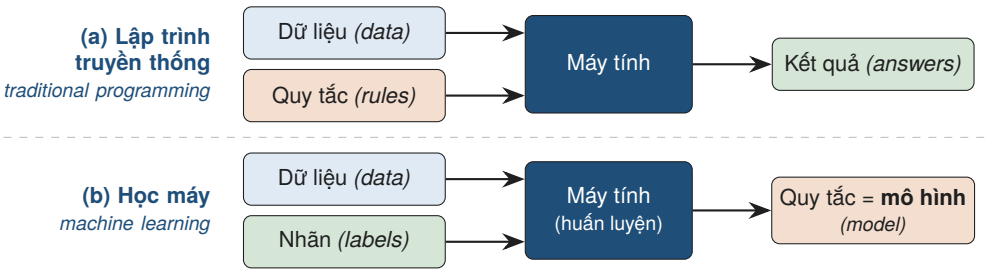
Không ai dạy em bé bằng định nghĩa “mèo là con vật bốn chân, có ria, tai nhọn, đuôi dài…” – định nghĩa đó vừa khó hiểu vừa sai (chó cũng bốn chân, đuôi dài!).

- Ta chỉ vào con mèo: “con mèo đấy”; gặp con chó: “con chó”.
- Sau vài chục lần, em bé **tự nhận ra** mèo – kể cả màu lông chưa từng thấy.
- Em bé đã **học từ ví dụ có đáp án**.

Học máy cũng vậy

Thay vì viết quy tắc, ta cho máy xem **rất nhiều ví dụ đã có đáp án** (*labeled examples*), máy tự rút ra cách phân biệt.

Lập trình truyền thống và học máy



- **Truyền thống**: con người viết quy tắc; máy áp quy tắc lên dữ liệu để cho ra kết quả.
- **Học máy**: con người đưa dữ liệu *cùng với* kết quả mong muốn; máy **tự rút ra quy tắc**.
- Trong học máy, quy tắc là **đầu ra** (*output*) chứ không phải đầu vào (*input*). Quy tắc học được gọi là **mô hình** (*model*).

Ví dụ nhỏ: đổi độ C sang độ F – cách 1, lập trình truyền thống

Ta **biết** công thức $F = 1{,}8 \times C + 32$ và viết nó thành chương trình:

ch01.py — ô c_sang_f_cong_thuc

```python
def c_sang_f(c):
    """Lập trình truyền thống: con người biết công thức và viết ra."""
    return c * 1.8 + 32

for c in [0, 25, 37, 100]:
    print(f"{c:>4} độ C = {c_sang_f(c):6.1f} độ F")
```

Kết quả

```
   0 độ C =   32.0 độ F
  25 độ C =   77.0 độ F
  37 độ C =   98.6 độ F
 100 độ C =  212.0 độ F
```

Câu hỏi cho lớp

Nếu ta **không biết** công thức, chỉ có một nhiệt kế ghi được cả hai thang đo, thì làm sao viết chương trình?

Cách 2 – học máy: cho máy tự tìm công thức

Độ C – *đặc trưng*	-10	0	8	15	22	30	38
Độ F – *nhãn*	14	32	46,4	59	71,6	86	100,4

ch01.py — ô c_sang_f_hoc

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Học máy: ta KHÔNG đưa công thức, chỉ đưa các cặp (đầu vào, đáp án) đã đo được
do_c = np.array([[-10], [0], [8], [15], [22], [30], [38]])   # đặc trưng (độ C)
do_f = np.array([14, 32, 46.4, 59, 71.6, 86, 100.4])         # nhãn (độ F)

mo_hinh_nhiet = LinearRegression()   # chọn một loại mô hình: đường thẳng
mo_hinh_nhiet.fit(do_c, do_f)        # HUẤN LUYỆN: máy tự tìm ra quy tắc
print("Hệ số máy tự tìm được:", round(mo_hinh_nhiet.coef_[0], 4))
print("Hằng số máy tự tìm được:", round(mo_hinh_nhiet.intercept_, 4))
print("Dự đoán cho 37 độ C:", round(mo_hinh_nhiet.predict([[37]])[0], 2), "độ F")
```

Kết quả

```
Hệ số máy tự tìm được: 1.8
Hằng số máy tự tìm được: 32.0
Dự đoán cho 37 độ C: 98.6 độ F
```

Máy tìm được $a = 1{,}8$, $b = 32$ – **đúng y công thức** mà ta chưa hề cho nó biết!

Ba điều rút ra từ ví dụ nhiệt độ

1. Ở cách 2, con người **không viết công thức**, chỉ cung cấp **dữ liệu có đáp án** (*labeled data*).
2. Con người vẫn phải chọn **loại** quy tắc cần tìm (ở đây: một đường thẳng, `LinearRegression`) – chọn **loại mô hình** (*model family*) là một phần quan trọng của học máy.
3. Lệnh `fit` là lúc máy "học" (*training*). Bạn sẽ gặp lại lệnh này trong **mọi chương**.

Lưu ý

Với bài toán đổi nhiệt độ, ta **nên** dùng công thức (cách 1): đơn giản, chính xác tuyệt đối. Học máy có giá trị khi ta **không thể** viết ra công thức – như bài toán thư rác.

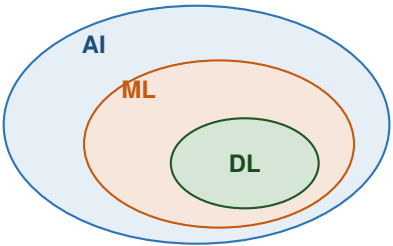
Định nghĩa học máy

Định nghĩa: Học máy – Machine Learning

Lĩnh vực nghiên cứu cách để máy tính **tự cải thiện** khả năng thực hiện một công việc nhờ **kinh nghiệm** (*experience*) rút ra từ dữ liệu, thay vì được **lập trình tường minh** (*explicitly programmed*) từng bước.

Vị trí của học máy

- **Trí tuệ nhân tạo** (*Artificial Intelligence, AI*): mọi kỹ thuật giúp máy tính thực hiện công việc "thông minh".
- **Học máy** (*Machine Learning, ML*): một nhánh của AI, máy *học từ dữ liệu*.
- **Học sâu** (*Deep Learning, DL*): một nhánh của học máy, dùng **mạng nơ-ron** (*neural network*) nhiều tầng (Chương 15).



Kiểm tra một bài toán bằng E – T – P

Định nghĩa của Tom Mitchell giúp kiểm tra một bài toán có phải học máy hay không:

> Một chương trình được gọi là **học** từ kinh nghiệm **E** đối với công việc **T** và thước đo **P** nếu hiệu quả làm công việc T, đo bằng P, **tăng lên nhờ kinh nghiệm E**.

Ví dụ: Phân tích bộ lọc thư rác

- **T** – công việc (*Task*): xếp mỗi thư vào "rác" hoặc "không rác".
- **E** – kinh nghiệm (*Experience*): 10 000 thư cũ đã được người dùng đánh dấu.
- **P** – thước đo (*Performance measure*): tỉ lệ % thư được xếp đúng loại.

Cho chương trình xem càng nhiều thư đã đánh dấu (E ↑) mà tỉ lệ xếp đúng càng cao (P ↑) $\Rightarrow$ chương trình đang **học**.

Câu hỏi cho lớp

Xác định T, E, P cho bài toán "dự đoán khách hàng có mua hàng sau khi xem quảng cáo hay không".

Bài tập mẫu: xác định T, E, P cho hệ thống gợi ý phim

Ví dụ đời thường: Học lái xe

T: lái xe an toàn. **E**: các giờ tập lái cùng thầy. **P**: số lỗi bị trừ điểm khi thi sát hạch. Tập càng nhiều (E ↑) mà càng ít lỗi (P tốt hơn) ⇒ đang học.

Bài tập mẫu: Ứng dụng xem phim muốn gợi ý phim cho từng người dùng

Lời giải.

- **T**: dự đoán người dùng có thích một bộ phim chưa xem hay không (để chọn phim đưa lên trang chủ).
- **E**: lịch sử xem của hàng triệu người – xem phim nào, xem hết hay bỏ giữa chừng, chấm mấy sao.
- **P**: tỉ lệ phim được gợi ý mà người dùng thật sự bấm xem (và xem hết).

Cùng một bài toán có thể chọn nhiều P khác nhau (ví dụ: tổng số phút xem mỗi tuần) – đó là quyết định của con người.

Khi nào nên (và không nên) dùng học máy?

Nên dùng khi

- **Quy tắc quá phức tạp** để con người viết ra: nhận diện khuôn mặt, đọc chữ viết tay, hiểu giọng nói (*speech recognition*).
- **Môi trường thay đổi liên tục**: thư rác, gian lận thẻ tín dụng (*fraud detection*), xu hướng mua sắm.
- **Có nhiều dữ liệu** chứa quy luật ẩn khó nhìn ra: dự đoán giá nhà, **gợi ý sản phẩm** (*recommendation*).

Không nên dùng khi

- Bài toán **đã có công thức rõ ràng**: tính tiền điện theo bậc thang, tính lương theo hợp đồng.
- **Không có dữ liệu** hoặc dữ liệu không đáng tin cậy.
- Cần kết quả **đúng tuyệt đối**, có thể kiểm chứng từng bước.

Lưu ý

Một câu lệnh `if` luôn dễ hiểu và dễ kiểm tra hơn một mô hình học máy.

Bài tập mẫu: có nên dùng học máy không?

	Bài toán
(a)	Tính học phí theo số tín chỉ đăng ký
(b)	Dự đoán sinh viên có nguy cơ bỏ học để hỗ trợ sớm
(c)	Kiểm tra mật khẩu đủ 8 ký tự, có chữ hoa và chữ số
(d)	Phân loại ảnh lá lúa “khỏe” hay “bị bệnh đạo ôn”

Bài tập mẫu: có nên dùng học máy không?

	Bài toán	Kết luận
(a)	Tính học phí theo số tín chỉ đăng ký	**Không**: có công thức (tín chỉ $\times$ đơn giá).
(b)	Dự đoán sinh viên có nguy cơ bỏ học để hỗ trợ sớm	**Nên**: không có công thức, nhưng có dữ liệu nhiều khóa trước kèm đáp án.
(c)	Kiểm tra mật khẩu đủ 8 ký tự, có chữ hoa và chữ số	**Không**: ba quy tắc rõ ràng, viết `if` là xong.
(d)	Phân loại ảnh lá lúa “khỏe” hay “bị bệnh đạo ôn”	**Nên**: quy tắc nhìn ảnh quá phức tạp; có thể thu thập ảnh đã gắn nhãn.

Mẹo nhỏ

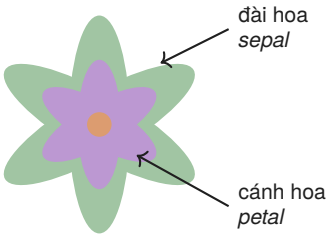
Hãy tự hỏi: **“Mình có viết ra được quy tắc không?”** Nếu viết được và quy tắc không đổi theo thời gian $\Rightarrow$ đừng dùng học máy.

Phần 2

Các thuật ngữ cơ bản

Bộ dữ liệu Iris

- Bộ **dữ liệu** (*dataset*) "kinh điển" nhất của học máy, do nhà thống kê **Ronald Fisher** công bố năm **1936**.
- Số đo của **150** bông hoa diên vĩ thuộc **3 loài**: *setosa*, *versicolor*, *virginica* (mỗi loài 50 bông).
- Mỗi bông đo **4 đại lượng** (cm):
  - chiều dài, chiều rộng **đài hoa** (*sepal length, sepal width*);
  - chiều dài, chiều rộng **cánh hoa** (*petal length, petal width*).
- Có sẵn trong `scikit-learn`: `load_iris()`.



(sơ đồ minh họa)

Ví dụ đời thường: sổ khám bệnh của phòng khám

Ví dụ đời thường: Mẫu – đặc trưng – nhãn trong đời sống

Phòng khám lưu hồ sơ thành một bảng: mỗi **dòng** là một bệnh nhân; các **cột** là tuổi, chiều cao, cân nặng, huyết áp, đường huyết; cột cuối là kết luận của bác sĩ "có / không mắc tiểu đường".

	đặc trưng *(features)*				**nhãn** *(label)*
	Tuổi	Cân nặng	Huyết áp	Đường huyết	Tiểu đường?
mẫu 1 *(sample)*	45	62	130	7,2	có
mẫu 2	30	55	115	5,1	không
mẫu 3	58	80	145	8,0	có

Cách nhớ: đặc trưng là thứ **dễ đo** (cân, đo huyết áp); nhãn là thứ **khó biết** mà ta muốn máy đoán giúp (phải là bác sĩ mới kết luận được).

Năm bông hoa của Iris trong Python

ch01.py — ô iris_nam_bong

```python
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
df = iris.frame
df["loai"] = iris.target_names[df["target"]]   # thêm cột tên loài cho dễ đọc
print(df.iloc[[0, 1, 50, 100, 149]])            # 5 bông hoa ở Bảng 1.1
```

Kết quả

```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                  5.1               3.5                1.4               0.2
1                  4.9               3.0                1.4               0.2
50                 7.0               3.2                4.7               1.4
100                6.3               3.3                6.0               2.5
149                5.9               3.0                5.1               1.8

     target        loai
0         0      setosa
1         0      setosa
50        1  versicolor
100       2   virginica
149       2   virginica
```

Bảng quá rộng nên pandas in thành hai phần; Python đếm từ **0**, nên bông hoa thứ 1 có chỉ số 0.

Mẫu, đặc trưng và nhãn

	Đặc trưng *(features)* x – đơn vị cm				**Nhãn** *(label)* y
Mẫu *(sample)*	Dài đài *sepal length*	Rộng đài *sepal width*	Dài cánh *petal length*	Rộng cánh *petal width*	Loài *species*
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
51	7.0	3.2	4.7	1.4	versicolor
101	6.3	3.3	6.0	2.5	virginica
150	5.9	3.0	5.1	1.8	virginica

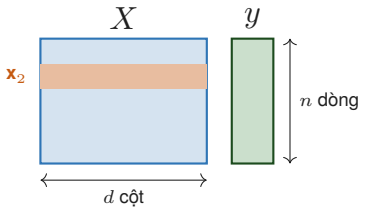
Định nghĩa: Ba khái niệm nền tảng

- **Mẫu** (*sample, instance, observation*): một đối tượng cần xử lý = **một dòng** của bảng. Mỗi bông hoa là một mẫu.
- **Đặc trưng** (*feature, attribute, input variable*): một thuộc tính đo được = **một cột đầu vào**. Iris có 4 đặc trưng.
- **Nhãn** (*label, target, output*): đáp án ta muốn máy dự đoán. Ở đây là **tên loài hoa**.

Ký hiệu toán học dùng trong suốt môn học

Toán dễ hiểu: n, d, X, $\mathbf{x}_i$, y

- n: **số mẫu** (*number of samples*) = số dòng. Iris: $n = 150$.
- d: **số đặc trưng** (*number of features*) = số cột đầu vào. Iris: $d = 4$.
- X: **ma trận đặc trưng** (*feature matrix*), kích thước $n \times d$ (đọc "n nhân d"). Iris: 150×4.
- $\mathbf{x}_i$: dòng thứ i của X – số đo của bông hoa thứ i: $\mathbf{x}_1 = (5.1,\ 3.5,\ 1.4,\ 0.2)$.
- y: **vector nhãn** (*target vector*) gồm n giá trị; $y_1 =$ setosa.



Chữ **in hoa** X: cả **bảng**; chữ **thường** y: **một cột**. Python dùng đúng quy ước này: x, y.

Luyện ký hiệu trên bảng năm căn hộ

ch01.py — ô bang_nha

```python
import pandas as pd
pd.set_option("display.max_columns", None)  # hiện đủ mọi cột khi in bảng
pd.set_option("display.width", 85)          # in gọn trong khổ 85 ký tự

nha = pd.DataFrame({
    "dien_tich": [45, 60, 72, 90, 120],       # m²
    "so_phong":  [1, 2, 2, 3, 4],
    "cach_tt":   [8, 5, 6, 3, 10],            # km đến trung tâm
    "gia":       [1.6, 2.4, 2.7, 4.1, 4.3]})  # tỷ đồng  <- NHÃN
print(nha)
```

Kết quả

```
   dien_tich  so_phong  cach_tt  gia
0         45         1        8  1.6
1         60         2        5  2.4
2         72         2        6  2.7
3         90         3        3  4.1
4        120         4       10  4.3
```

Bài tập mẫu: Đọc ký hiệu

Dự đoán `gia` (tỷ đồng) từ 3 cột còn lại. Cho biết n, d, cỡ X, $\mathbf{x}_3$, y_3, y_5.

Lời giải.

- 5 dòng $\Rightarrow n = 5$; 3 cột đầu vào $\Rightarrow d = 3$.
- X cỡ 5×3; y gồm 5 giá trị.
- $\mathbf{x}_3 = (72,\ 2,\ 6),\ \ y_3 = 2{,}7$ tỷ.
- $y_5 = 4{,}3$ tỷ (căn 120 m²).

Tách X, y bằng pandas

ch01.py — ô bang_nha_X_y

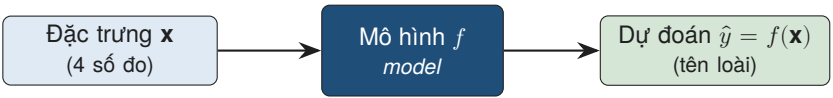
```
X_nha = nha[["dien_tich", "so_phong", "cach_tt"]]   # 3 cột đặc trưng
y_nha = nha["gia"]                                  # 1 cột nhãn
n, d = X_nha.shape
print("n (số mẫu) =", n, "  d (số đặc trưng) =", d)
print("Mẫu thứ 3, x_3 =", X_nha.iloc[2].tolist())   # Python đếm từ 0
print("Nhãn thứ 3, y_3 =", y_nha.iloc[2])
```

Kết quả

```
n (số mẫu) = 5   d (số đặc trưng) = 3
Mẫu thứ 3, x_3 = [72, 2, 6]
Nhãn thứ 3, y_3 = 2.7
```

- X: lấy **danh sách** tên cột – **hai** cặp ngoặc vuông `nha[[...]]`.
- y: lấy **một** cột – **một** cặp ngoặc `nha["gia"]`.
- `X_nha.shape` trả về cặp `(số dòng, số cột)` $= (n, d)$.

Mô hình, huấn luyện và dự đoán



Định nghĩa: Ba khái niệm của quá trình học

- **Mô hình** (*model*): "quy tắc" máy học được – một **hàm** f nhận đặc trưng **x**, trả về dự đoán $\hat{y} = f(\mathbf{x})$. Dấu mũ trên $\hat{y}$ (đọc "y mũ", *y-hat*) phân biệt **giá trị dự đoán** với **giá trị thật** y.
- **Huấn luyện** (*training, fitting*): dùng dữ liệu **đã có nhãn** để tìm ra mô hình f. Trong `scikit-learn`: `.fit()`.
- **Dự đoán** (*prediction, inference*): dùng mô hình đã huấn luyện để đưa ra đáp án cho **mẫu mới**. Trong `scikit-learn`: `.predict()`.

Ví dụ đời thường: học sinh ôn thi

Ví dụ đời thường: Mô hình là một học sinh

- **Huấn luyện** = học sinh luyện các đề cũ **có đáp án**; làm xong so đáp án, sai đâu sửa đấy.
- **Dự đoán** = học sinh vào phòng thi gặp **đề mới** (không có đáp án) và tự làm.
- **Mô hình** = kiến thức, mẹo làm bài học sinh tích lũy được.

Trong ví dụ đổi nhiệt độ: mô hình là $f(C) = 1{,}8C + 32$; huấn luyện là `fit` trên 7 cặp số; dự đoán là `predict([[37]])` $\Rightarrow \hat{y} = 98{,}6$.

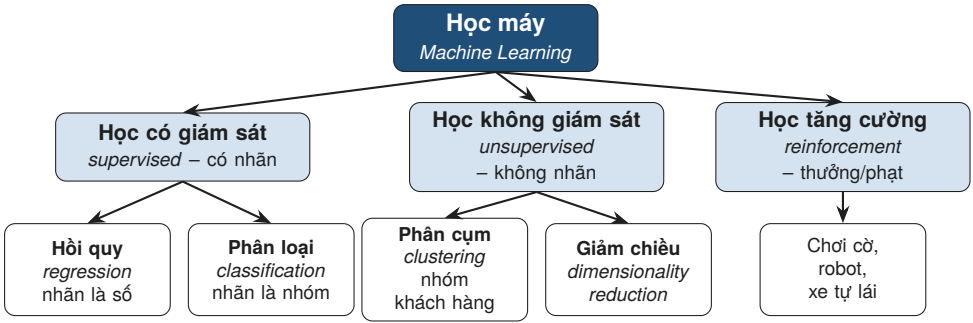
Câu hỏi cho lớp

Ngân hàng có bảng 2 000 khách vay với các cột thu nhập, tuổi, số khoản nợ, và cột "trả nợ đúng hạn?". Chỉ ra mẫu, đặc trưng, nhãn, n, d.

Phần 3

Các nhóm bài toán học máy

Ba nhóm học máy chính



Cách chia dựa vào câu hỏi: **dữ liệu có nhãn hay không**, và máy nhận "tín hiệu học" từ đâu.

Học có giám sát: hồi quy và phân loại

Định nghĩa: Học có giám sát – Supervised learning

Học từ dữ liệu **có nhãn**: mỗi mẫu đi kèm đáp án đúng – như **luyện đề có đáp án**; "người giám sát" chính là đáp án.

Hai loại bài toán, phân biệt bằng **kiểu của nhãn**:

Hồi quy (*regression*)

Nhãn là **con số liên tục**, có thể nhận vô số giá trị. Ví dụ: giá nhà (2,35 tỷ; 3,1 tỷ…), nhiệt độ ngày mai, doanh thu tháng tới.

Phân loại (*classification*)

Nhãn là **một nhóm** (*class*) trong số ít nhóm cho trước. Ví dụ: thư rác / không rác (2 lớp, *binary*); loài hoa, chữ số 0–9 (nhiều lớp, *multi-class*).

Mẹo nhỏ

Hỏi "*Câu trả lời có thể là 2,5 không?*". Hợp lý (giá nhà 2,5 tỷ) ⇒ **hồi quy**; vô nghĩa (loài hoa thứ 2,5) ⇒ **phân loại**.

Phân loại nhị phân, nhiều lớp – và một bẫy thường gặp

- Mỗi nhóm trong bài toán phân loại gọi là một **lớp** (*class*).
- Đúng 2 lớp (rác/không rác, đậu/rớt): **phân loại nhị phân** (*binary classification*).
- Từ 3 lớp trở lên (3 loài hoa, 10 chữ số): **phân loại nhiều lớp** (*multiclass classification*).

Lưu ý

Đừng nhìn **kiểu dữ liệu** mà kết luận! Nhãn Iris lưu thành 0, 1, 2 nhưng chỉ là **mã** của loài hoa – "loài 1,5" vô nghĩa $\Rightarrow$ **phân loại**. Điểm thi lưu thành số và 7,5 có nghĩa $\Rightarrow$ **hồi quy**.

Học không giám sát và học tăng cường

Định nghĩa: Học không giám sát – Unsupervised

Học từ dữ liệu **không có nhãn**. Máy phải **tự tìm cấu trúc ẩn** trong dữ liệu.

- **Phân cụm** (*clustering*): cửa hàng có dữ liệu mua sắm của 10 000 khách nhưng không biết ai thuộc nhóm nào; thuật toán tự gom khách có thói quen giống nhau (Chương 13).
- **Giảm chiều** (*dimensionality reduction*): nén nhiều cột thành ít cột (Chương 14).

Định nghĩa: Học tăng cường – Reinforcement

Mô phỏng cách dạy thú cưng.

- **Tác tử** (*agent*) thử các **hành động** (*action*) trong một **môi trường** (*environment*).
- Nhận **phần thưởng** (*reward*) khi làm tốt, **hình phạt** khi làm sai.
- Dần học được cách hành động tốt nhất: chơi cờ, robot, xe tự lái.

Học tăng cường nằm ngoài phạm vi giáo trình.

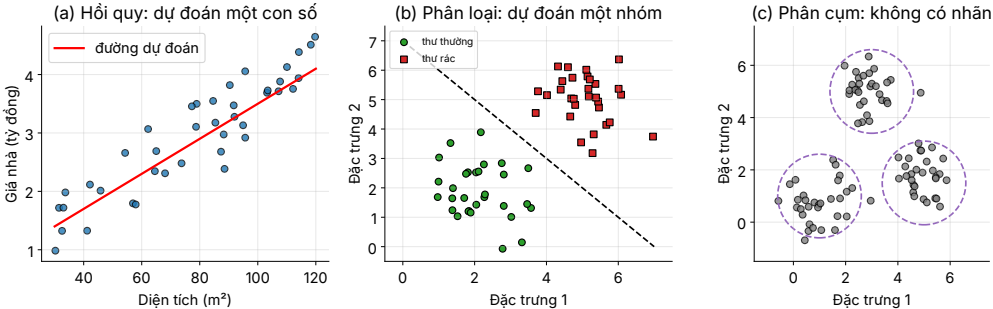
Ví dụ đời thường: sắp xếp tủ quần áo

Ví dụ đời thường: Phân cụm – không ai gắn nhãn trước

Bạn đổ hết quần áo ra giường để xếp vào tủ. Không ai cho trước danh sách "ngăn 1 là áo sơ mi, ngăn 2 là quần jean". Bạn tự nhìn và gom những món **giống nhau** lại: áo với áo, quần với quần, đồ thể thao một chồng.

- Đó chính là **phân cụm** (*clustering*): chia đồ vật thành nhóm **không cần nhãn**.
- Cửa hàng có dữ liệu mua sắm của 10 000 khách, không biết ai thuộc nhóm nào $\Rightarrow$ thuật toán tự gom khách có thói quen giống nhau (nhóm hay săn giảm giá, nhóm mua ít nhưng đắt…).
- **Giảm chiều** (*dimensionality reduction*): nén bảng trăm cột thành vài cột (Chương 13–14).

Minh họa trực quan: hồi quy – phân loại – phân cụm



- (a) **Hồi quy** (*regression*): tìm **đường thẳng** dự đoán giá nhà từ diện tích – đầu ra là một số.
- (b) **Phân loại** (*classification*): tìm **đường ranh giới** (*decision boundary*) tách hai loại thư.
- (c) **Phân cụm** (*clustering*): dữ liệu **không có nhãn** (mọi điểm cùng màu xám), máy tự tìm ra ba nhóm.

Cách đọc từng ô của hình minh họa

- **(a) Hồi quy**: trục ngang là **đặc trưng** (diện tích), trục dọc là **nhãn** (giá). Mô hình là đường đỏ; với căn nhà mới, dóng từ diện tích lên đường đỏ để đọc ra giá.
- **(b) Phân loại**: **cả hai trục** đều là đặc trưng; nhãn thể hiện bằng **màu và hình dạng** điểm. Mô hình là đường đứt nét chia mặt phẳng thành hai vùng (*decision boundary*).
- **(c) Phân cụm**: mọi điểm cùng màu xám vì **không có nhãn**; vòng tròn tím là nhóm thuật toán tự tìm ra.

Luyện tập: xác định loại bài toán

Tình huống	Loại bài toán
Dự đoán điểm thi cuối kỳ của sinh viên	
Dự đoán xếp loại học lực: giỏi, khá, trung bình, yếu	
Dự đoán số khách đến quán cà phê vào thứ Bảy	
Dự đoán sinh viên đậu hay rớt môn	
Nhận dạng chữ số viết tay 0–9	
Dự đoán lượng mưa ngày mai (mm)	
Chia 5 000 bài báo thành các chủ đề, chưa biết trước chủ đề	
Phát hiện giao dịch thẻ tín dụng gian lận	

Luyện tập: xác định loại bài toán

Tình huống	Loại bài toán
Dự đoán điểm thi cuối kỳ của sinh viên	Hồi quy *(regression)*
Dự đoán xếp loại học lực: giỏi, khá, trung bình, yếu	Phân loại 4 lớp
Dự đoán số khách đến quán cà phê vào thứ Bảy	Hồi quy
Dự đoán sinh viên đậu hay rớt môn	Phân loại 2 lớp *(binary)*
Nhận dạng chữ số viết tay 0–9	Phân loại 10 lớp *(multi-class)*
Dự đoán lượng mưa ngày mai (mm)	Hồi quy
Chia 5 000 bài báo thành các chủ đề, chưa biết trước chủ đề	Phân cụm *(clustering)*
Phát hiện giao dịch thẻ tín dụng gian lận	Phân loại 2 lớp

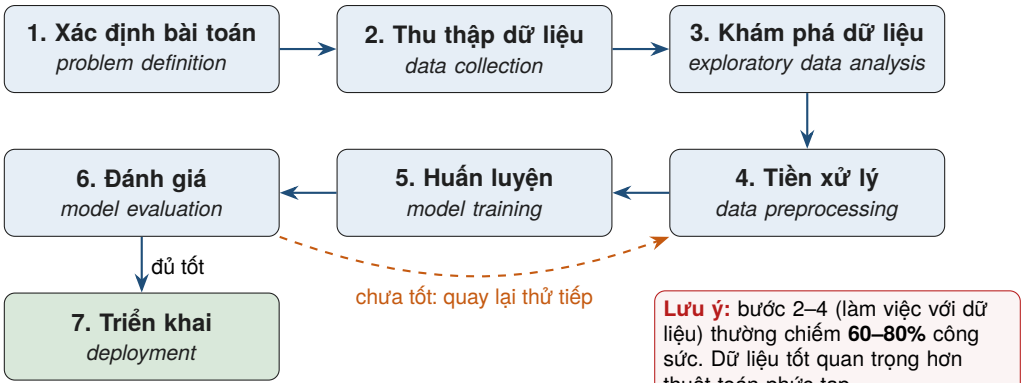
Lưu ý

Hai dòng đầu: **cùng dữ liệu** về sinh viên, nhưng **câu hỏi khác nhau** dẫn đến loại bài toán khác nhau. Loại bài toán do **câu hỏi** quyết định, không do dữ liệu.

Phần 4

Quy trình một dự án học máy

Bảy bước của một dự án học máy



Quy trình mang tính **lặp** (*iterative*): kết quả chưa tốt thì quay lại xử lý dữ liệu hoặc thử mô hình khác.

Nội dung từng bước

1. **Xác định bài toán**: muốn dự đoán gì? Hồi quy hay phân loại? Dùng **thước đo** (*metric*) nào để biết mô hình tốt (T, E, P)?
2. **Thu thập dữ liệu**: từ cơ sở dữ liệu doanh nghiệp, khảo sát, cảm biến, hoặc bộ dữ liệu có sẵn như trong giáo trình.
3. **Khám phá dữ liệu** (*EDA*): bao nhiêu dòng, cột; giá trị trung bình; vẽ biểu đồ để tìm quy luật ban đầu (Chương 4).
4. **Tiền xử lý**: xử lý **giá trị thiếu** (*missing values*), đổi chữ thành số (*encoding*), đưa các cột về cùng thang đo (*scaling*) (Chương 4).
5. **Huấn luyện**: chọn **thuật toán** (*algorithm*) và cho mô hình học từ dữ liệu huấn luyện.
6. **Đánh giá**: đo hiệu quả của mô hình trên dữ liệu **mà nó chưa từng thấy** (*unseen data*).
7. **Triển khai**: tích hợp mô hình vào ứng dụng thực tế và **theo dõi** (*monitoring*) trong quá trình sử dụng.

Bảy bước qua một dự án cụ thể: dự đoán giá căn hộ

Bước	Công việc cụ thể
1. Xác định bài toán	Dự đoán giá bán (tỷ đồng) $\Rightarrow$ hồi quy. P: sai lệch trung bình so với giá thật, mong muốn dưới 10%.
2. Thu thập dữ liệu	20 000 giao dịch trong 3 năm: diện tích, số phòng, tầng, quận, năm xây, giá bán.
3. Khám phá dữ liệu	Giá trung bình mỗi quận? Diện tích lớn thì giá có tăng đều? Có căn ghi 0 m^2 (lỗi nhập liệu)?
4. Tiền xử lý	Xóa/sửa dòng lỗi, điền chỗ trống, đổi tên quận (chữ) thành số.
5. Huấn luyện	Thử vài loại mô hình (hồi quy tuyến tính, cây quyết định…) trên 80% giao dịch.
6. Đánh giá	Dự đoán 20% giao dịch còn lại, so với giá thật. Sai 15%? Quay lại bước 4 hoặc 5.
7. Triển khai	Đưa lên trang web cho nhân viên dùng; mỗi quý cho mô hình học lại vì giá nhà thay đổi.

Ví dụ đời thường: Nấu một món ăn

Xác định bài toán = chọn món; thu thập = đi chợ; khám phá, tiền xử lý = nhặt, rửa, thái, ướp (lâu nhất!); huấn luyện = nấu; đánh giá = nếm; triển khai = dọn ra bàn.

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Học máy	Machine learning
Lập trình truyền thống	Traditional programming
Quy tắc	Rule
Trí tuệ nhân tạo	Artificial intelligence
Bộ dữ liệu	Dataset
Mẫu	Sample / instance
Đặc trưng	Feature
Nhãn	Label / target
Mô hình	Model

Tiếng Việt	Tiếng Anh
Huấn luyện	Training (fit)
Dự đoán	Prediction (predict)
Học có giám sát	Supervised learning
Học không giám sát	Unsupervised learning
Học tăng cường	Reinforcement learning
Hồi quy	Regression
Phân loại nhị phân / nhiều lớp	Binary / multiclass
Phân cụm	Clustering
Giảm chiều	Dimensionality reduction

Phần 5

Tổng kết Buổi 1.1

Tóm tắt Buổi 1.1

- Học máy giúp máy tính **tự rút ra quy tắc từ dữ liệu có đáp án**; quy tắc học được gọi là **mô hình** (*model*).
- Mô tả một bài toán bằng **T – E – P**. Không dùng học máy khi đã có công thức rõ ràng hoặc không có dữ liệu.
- Dữ liệu = **mẫu** (*samples*) (dòng) + **đặc trưng** (*features*) (X, cỡ $n \times d$) + **nhãn** (*labels*) (y).
- **Có giám sát**: **hồi quy** (*regression*) (nhãn là số), **phân loại** (*classification*) (nhãn là nhóm).
  Không giám sát: **phân cụm** (*clustering*), giảm chiều.
- Dự án học máy đi qua **7 bước**; làm việc với dữ liệu chiếm phần lớn thời gian.

Câu hỏi thảo luận

1. Trình bày sự khác nhau giữa lập trình truyền thống và học máy. Vẽ lại sơ đồ minh họa.
2. Với bài toán "dự đoán khách hàng có mua hàng sau khi xem quảng cáo hay không", xác định T, E, P.
3. Trong Iris, cho biết n, d, kích thước của X và y. Một mẫu tương ứng với gì ngoài thực tế?
4. Hồi quy, phân loại hay phân cụm? (a) số lượt xem của một video; (b) ảnh chó hay mèo; (c) gom bài hát có giai điệu giống nhau khi chưa có thể loại; (d) số sao 1–5 của một bình luận.
5. Vì sao trong học máy, con người vẫn phải chọn **loại** mô hình dù không viết quy tắc?

Bài tập Buổi 1.1

- ⋆ **1.1**: Liệt kê năm ứng dụng học máy bạn gặp hằng ngày; mỗi ứng dụng là bài toán gì, nhãn là gì?
- ⋆ **1.2**: Thêm hai thư rác mà `la_thu_rac` không nhận ra; sửa hàm để nhận ra. Có chắc không nhầm thư thường?
- ⋆ **1.3**: Trong `c_sang_f_hoc`, chỉ giữ hai cặp số đầu rồi huấn luyện lại; thử thêm sai số (46,4 $\to$ 47).
- ⋆ **1.4**: Tạo DataFrame 6 bạn (giờ tự học, số buổi vắng, điểm giữa kỳ, điểm cuối kỳ); tách X, y, in n, d.
- ⋆⋆ **1.5**: Nghĩ ra một bài toán học máy cho nông nghiệp, giao thông, du lịch, thể thao; xác định T, E, P.

Chi tiết: *Bài tập Buổi 1.1*, Chương 1. Buổi sau: **Buổi 1.2 – Chương trình học máy đầu tiên** (hãy chuẩn bị tài khoản Google để dùng Colab).